ASP.Net Core Project Structure

Overview

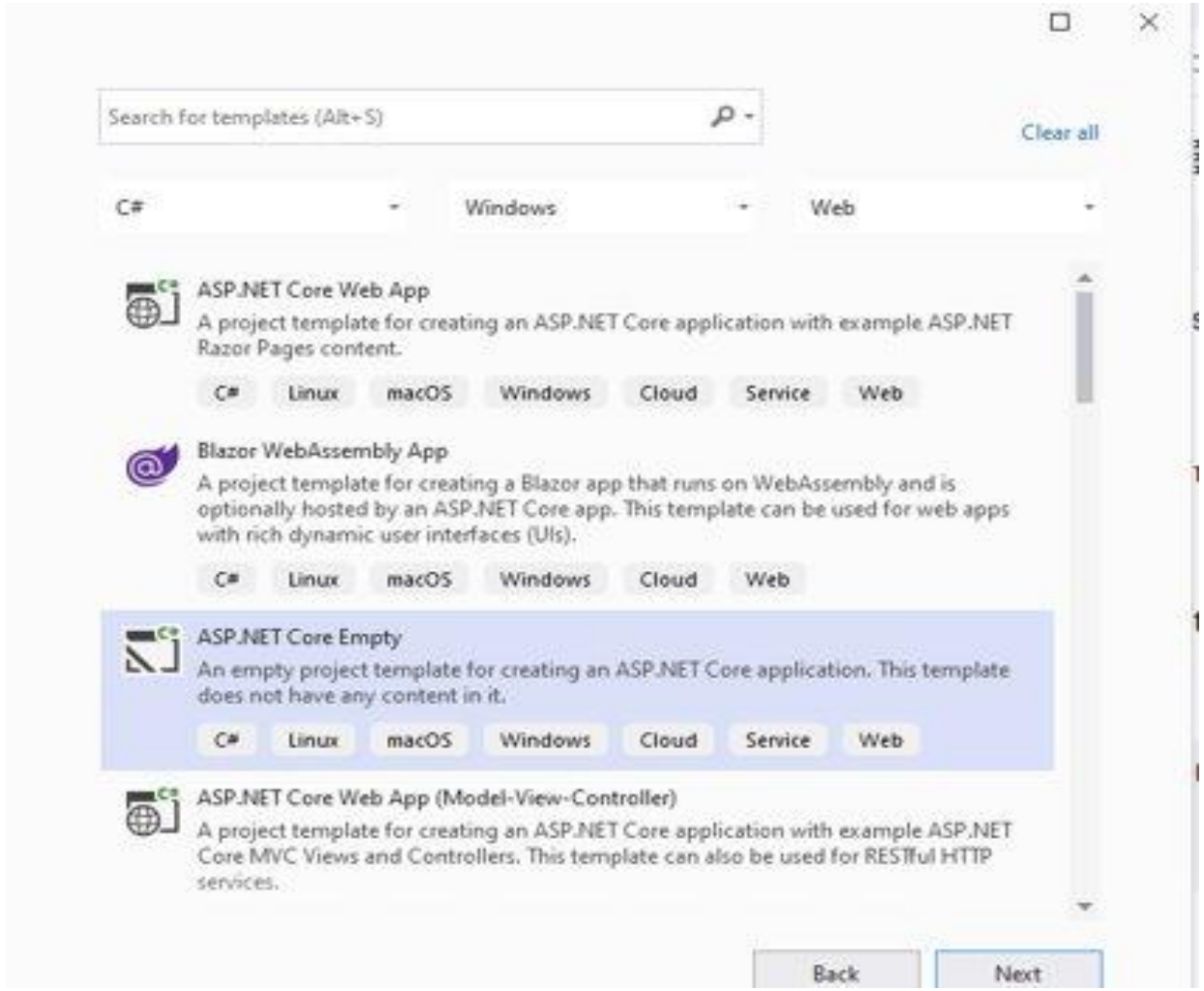


How to create ASP.Net Core Application using VS?

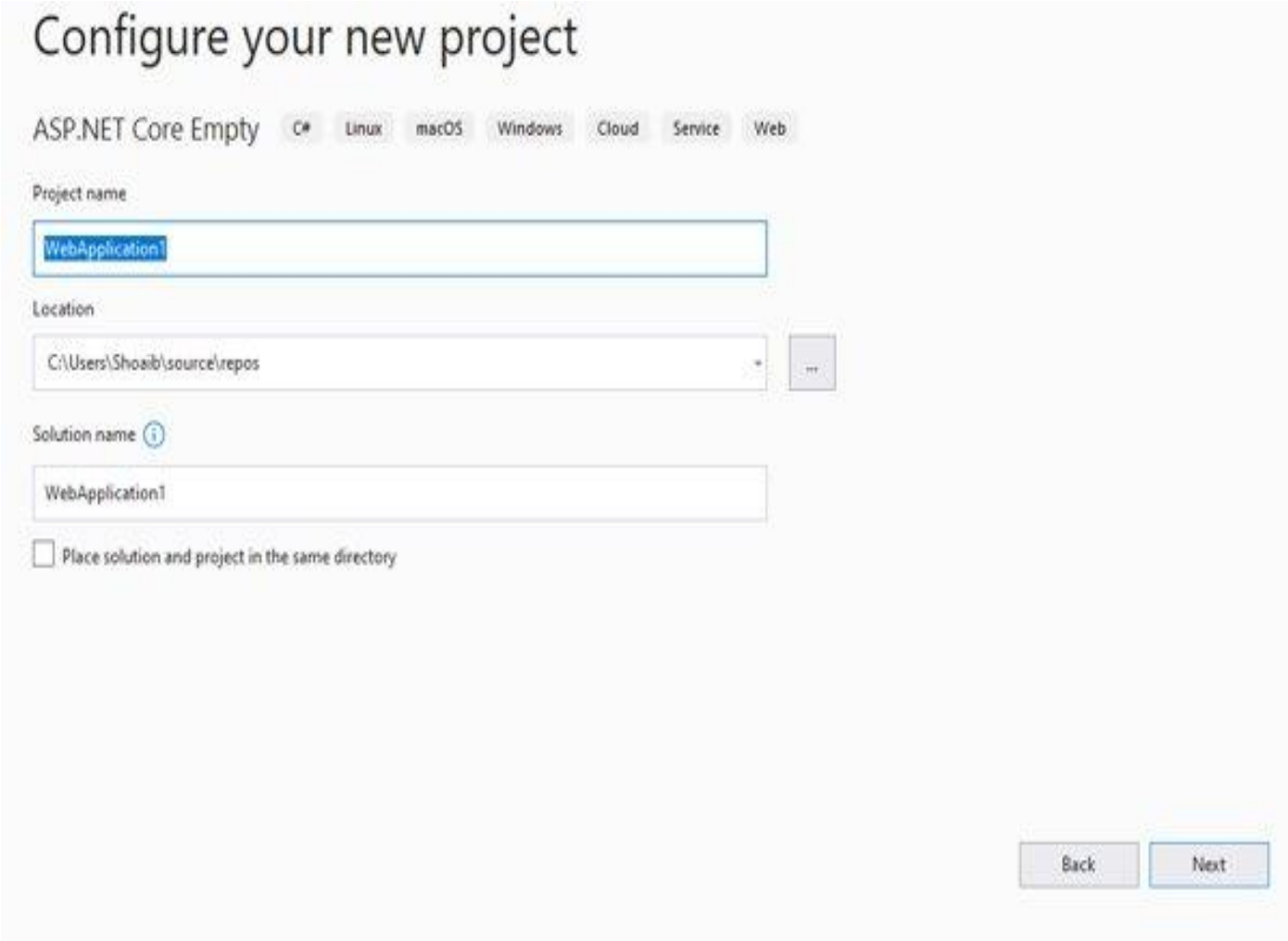
How to create ASP.Net Core Application CLI?

How to create ASP.Net Core Application using VS?

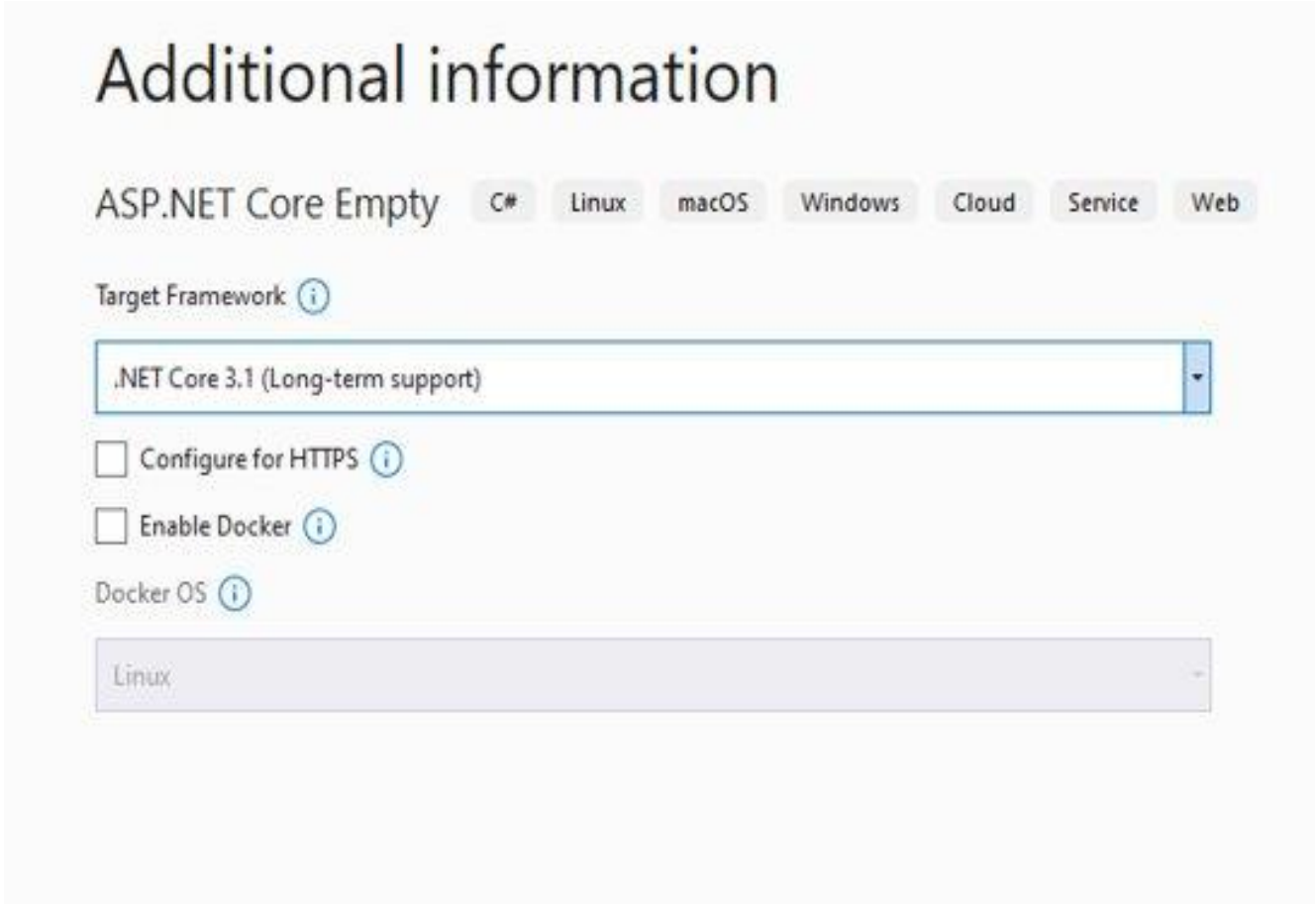
Step1:


Search for templates (Alt+S)
Clear all
C#
Windows
Web
ASP.NET Core Web App
A project template for creating an ASP.NET Core application with example ASP.NET Razor Pages content.
C# Linux macOS Windows Cloud Service Web
Blazor WebAssembly App
A project template for creating a Blazor app that runs on WebAssembly and is optionally hosted by an ASP.NET Core app. This template can be used for web apps with rich dynamic user interfaces (UIs).
C# Linux macOS Windows Cloud Web
ASP.NET Core Empty
An empty project template for creating an ASP.NET Core application. This template does not have any content in it.
C# Linux macOS Windows Cloud Service Web
ASP.NET Core Web App (Model-View-Controller)
A project template for creating an ASP.NET Core application with example ASP.NET Core MVC Views and Controllers. This template can also be used for RESTful HTTP services.
Back
Next

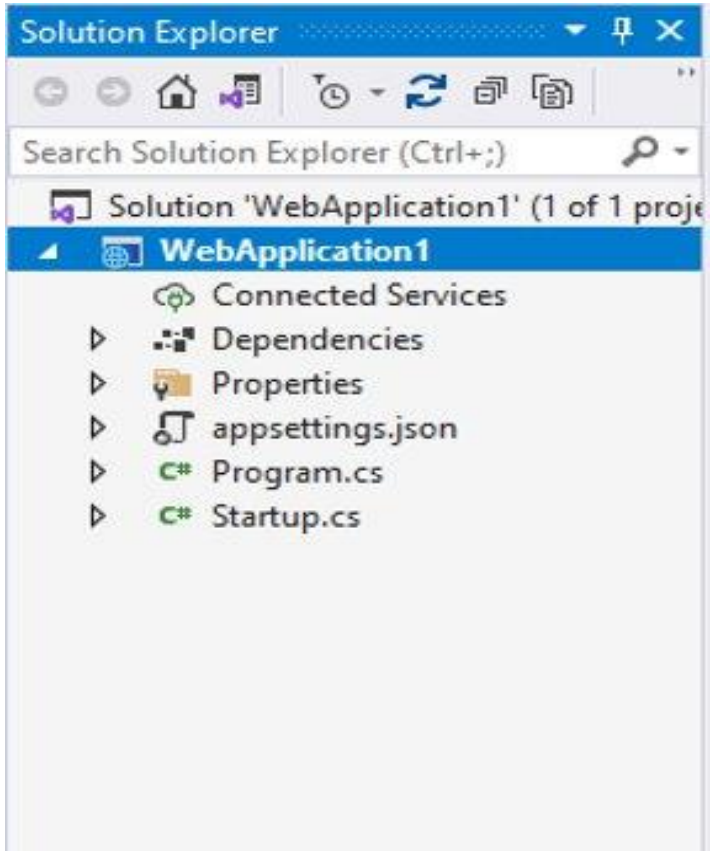
Step2:


Configure your new project
ASP.NET Core Empty
C#
Linux
macOS
Windows
Cloud
Service
Web
Project name
WebApplication1
Location
C:\Users\Shoaib\source\repos
...
Solution name
WebApplication1
Place solution and project in the same directory
Back
Next

Step3:


Additional information
ASP.NET Core Empty
C#
Linux
macOS
Windows
Cloud
Service
Web
Target Framework
.NET Core 3.1 (Long-term support)
Configure for HTTPS
Enable Docker
Docker OS
Linux

Project Files:



How to create ASP.Net Core Application using CLI?

To create an ASP.NET Core web application with a specific version of .NET Core, you can use the .NET Core CLI (Command-Line Interface). Here's how you can create an ASP.NET Core web app with a specific version of .NET Core:

- **Open a Command Prompt or Terminal:**

-Open a command prompt on Windows or a terminal on macOS or Linux.

- **Navigate to the Directory Where You Want to Create the Project:**

-Use the cd command to navigate to the directory where you want to create your ASP.NET Core web app.

- **Create the ASP.NET Core Web App:**

-Use the dotnet new command to create a new ASP.NET Core web application. You need to specify the appropriate template for the type of application you want to create. You also need to use the --framework flag to specify the target framework version.

https://learn.microsoft.com/en-us/dotnet/core/tools/dotnet

For example, if you want to create a new ASP.NET Core web application using .NET Core 3.1, you would run:

```
dotnet new web --framework netcoreapp3.1 -n MyWebApp
```

- Navigate to the Project Directory:

-Navigate to the newly created project directory using the cd command. In this example, you would run:

```
cd MyWebApp
```

- Restore Dependencies:

-Use the dotnet restore command to restore the project dependencies:

```
dotnet restore
```

- Build and Run the Application:

-Build and run the application using the dotnet run command:

```
dotnet run
```

Project config file:

Target Framework Moniker (TFM)

```
WebApplication1.csproj*
1  <Project Sdk="Microsoft.NET.Sdk.Web">
2
3    <PropertyGroup>
4      <TargetFramework>netcoreapp3.1</TargetFramework>
5      <AspNetCoreHostingModel>InProcess</AspNetCoreHostingModel>
6    </PropertyGroup>
7
8  </Project>
9
```

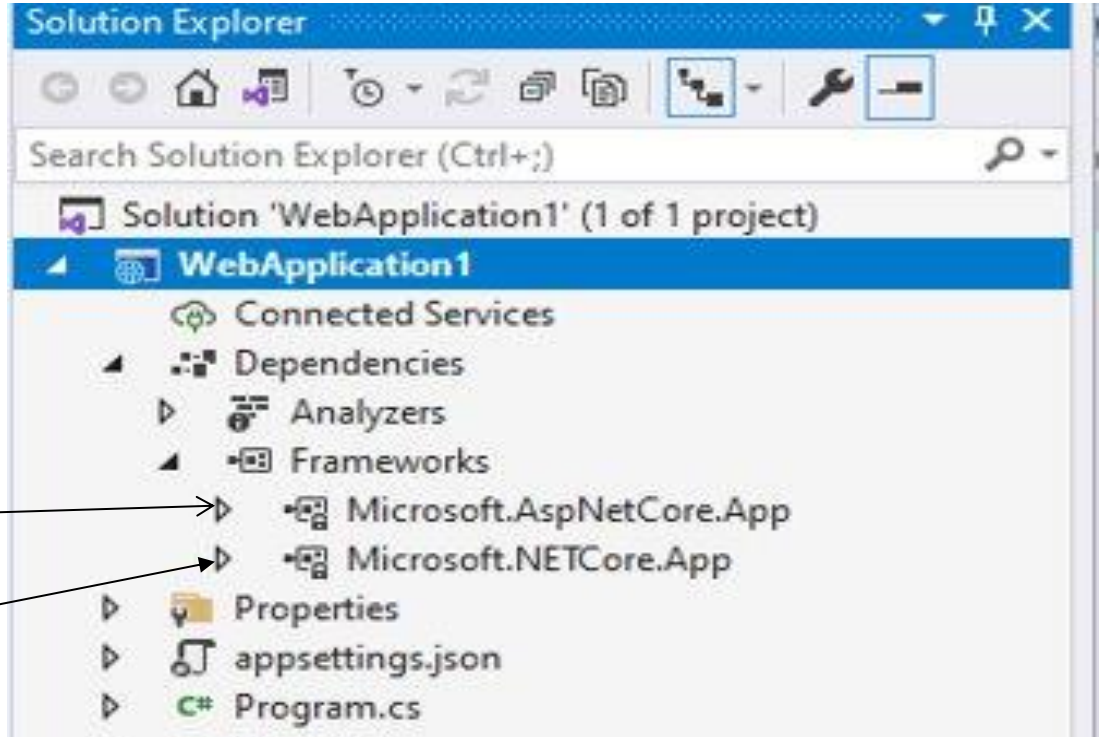
Name	Abbreviation	TFM
.NET Framework	net	net451
		net472
.NET Core	netcoreapp	netcoreapp1.0
		netcoreapp2.2

-Hosting Models

InProcess: IIS Express

OutOfProcess: Kestrel Server

Dependencies :



ASP.Net Core Meta Package

.Net Core Meta Package

Program.cs

In .NET 6, the Program.cs file is the entry point of the application. It defines the application's startup logic, including dependency injection, middleware configuration, and service registration.

Key Features of Program.cs in .NET 6:

- **Minimal Hosting Model:** .NET 6 introduced a simplified way to configure and run applications.
- **Top-Level Statements:** Unlike earlier versions, the Main method is omitted, making the code more concise.
- **Web Application Configuration:** It is used to set up services and middleware for ASP.NET Core applications.

Program.cs

```
var builder = WebApplication.CreateBuilder(args);

// Add services to the container

builder.Services.AddControllers();

var app = builder.Build();

// Configure the HTTP request pipeline

app.MapControllers();

app.Run();
```

Breakdown of the Code:

WebApplication.CreateBuilder(args);

- Initializes the application builder.
- Configures services, logging, and configuration sources.

builder.Services.AddControllers();

Adds MVC controllers to handle HTTP requests.

app.MapControllers();

Maps controller endpoints for handling.

app.Run();Starts the application and begins listening for incoming requests.

References

https://learn.microsoft.com/en-us/aspnet/core/host-and-deploy/directory-structure

https://www.tutorialsteacher.com/core/aspnet-core-application-project-structure

https://www.c-sharpcorner.com/article/folder-structure-of-asp-net-core-mvc-6-0-project/

https://www.tutorialspoint.com/asp.net_core/asp.net_core_project_layout.htm

https://www.tutorialspoint.com/asp.net_core/asp.net_core_project_layout.htm

https://jayanttripathy.com/asp-net-core-mvc-project-structure-and-process-flow/

http://www.mtutorial.com/asp.net-core-project-and-folder-structure